

KANBAN STUDIO PRO ■

A-Z Comprehensive Architectural Documentation, File Reference & Deployment Guide

1. Executive Summary & Application Overview

Kanban Studio Pro is a full-stack, enterprise-grade project management web application. It features dynamic drag-and-drop task boards, multi-project workspace isolation, granular Role-Based Access Control (RBAC), real-time WebSocket synchronization across devices, and an embedded natural-language AI Kanban Assistant.

2. Complete Technology Stack

Layer	Technologies & Version	Purpose / Key Features
Frontend	Next.js 16 (Turbopack), React 19, TypeScript, TailwindCSS v4	High-performance App Router SPA with server/client components and modern glassmorphism styling.
Drag & Drop	@dnd-kit/core, @dnd-kit/sortable	Accessible, touch-friendly task reordering with MouseSensor, PointerSensor & TouchSensor (150ms delay).
Backend API	FastAPI, Python 3.13, Uvicorn ASGI Server	Asynchronous REST API endpoints, WebSocket connection manager, rate limiting, and CORS routing.
Database	SQLite3 (WAL Mode), Python sqlite3 module	Relational persistence for users, projects, columns, cards, member roles, activity logs, and notifications.
AI Assistant	Custom Rule Engine & Google Gemini API	Parses natural language prompts to automatically execute structured board mutations (add, move, clear tasks).
DevOps & Cloud	Docker, Render.com, Vercel, GitHub Actions	Containerized Python backend on Render + static Edge CDN frontend hosting on Vercel.
Testing	Pytest (22), Vitest (44), Playwright E2E (14)	Comprehensive 80-test automated suite covering backend API, frontend UI, state logic, and browser flows.

3. Complete Directory & File Structure (A-Z Guide)

Below is a breakdown of every single file in the repository and its exact technical responsibility:

- **Dockerfile (Root):** Multi-stage Python 3.13 container definition for building and running the FastAPI backend on cloud hosts (Render/Railway). Installs system dependencies, copies requirements, and launches uvicorn on port 8000.

- **render.yaml (Root):** Render Blueprint deployment configuration file. Configures automatic container builds from root Dockerfile and passes CORS_ORIGINS and PORT environment variables.
- **README.md (Root):** Project documentation detailing live demo links, architecture overview, screenshot previews, and setup instructions.
- **pm/backend/main.py:** FastAPI application entry point. Defines REST endpoints (/api/auth/register, /api/auth/login, /api/projects, /api/board, /api/cards, /api/ai/chat) and WebSocket route (/ws/projects/{id}).
- **pm/backend/database.py:** Core SQLite database layer. Contains schema definitions, table migrations, connection management, PBKDF2 password hashing, RBAC permission checks, CRUD operations, and default board seeding.
- **pm/backend/ai.py:** AI Assistant logic. Integrates Google Gemini API with fallbacks to execute structured board mutations (creating tasks, moving cards across columns, clearing completed items).
- **pm/backend/websocket_manager.py:** ConnectionManager class managing real-time WebSocket client connections and broadcasting board updates to connected project members.
- **pm/backend/config.py:** Environment configuration loader for reading database paths, CORS origin lists, and API keys.
- **pm/backend/requirements.txt:** Backend Python dependencies list (fastapi, uvicorn, pydantic, pytest, google-generativeai).
- **pm/backend/test_main.py & test_database.py:** Pytest backend test suite (22 tests) validating authentication, DB queries, card mutations, RBAC access checks, and AI response payloads.
- **pm/frontend/src/app/page.tsx & layout.tsx:** Next.js App Router root layout and primary page shell rendering the main Kanban interface.
- **pm/frontend/src/app/globals.css:** Global TailwindCSS v4 stylesheet containing modern theme CSS variables, glassmorphism card utilities, glowing animations, and vertical mobile column layout rules.
- **pm/frontend/src/components/KanbanBoard.tsx:** Core frontend board orchestrator. Manages user auth state, project switcher, undo/redo history, WebSocket real-time sync, @dnd-kit sensors, and column grid.
- **pm/frontend/src/components/KanbanColumn.tsx:** Droppable column container component rendering column headers, task counter badges, card lists, and quick-add task buttons.
- **pm/frontend/src/components/KanbanCard.tsx:** Sortable task card component with smooth hover micro-animations, priority tags, due dates, assignees, and quick edit/delete buttons.
- **pm/frontend/src/components/AIAssistantWidget.tsx:** Floating AI Assistant chat drawer. Connects to /api/ai/chat via getApiUrl and automatically applies server-returned board updates.
- **pm/frontend/src/components/LoginForm.tsx:** Sign-in and Create Account tabbed modal featuring auto-login, error alerts, case normalization, and fallback client registration.
- **pm/frontend/src/lib/api.ts:** API client module exporting getApiUrl, fetchBoard, saveBoard, registerApi, and project management network handlers.
- **pm/frontend/src/lib/kanban.ts:** Kanban data structures, immutability helpers, moveCard logic, and initial default board generator.
- **pm/frontend/src/lib/useUndoRedo.ts:** Custom React hook providing multi-level Undo (Ctrl+Z) and Redo (Ctrl+Y) state history management.

- **pm/frontend/tests/kanban.spec.ts:** Playwright E2E browser test suite (14 tests) running automated user interactions across desktop and mobile viewports.

4. Core Site Logic & Key Functionalities

Authentication & Case Normalization: Users can sign in or register new accounts. Username strings are automatically sanitized and lower-cased to prevent case-sensitivity mismatches during authentication. Passwords are salted and hashed using PBKDF2-HMAC-SHA256.

Multi-Project Isolation: Users can create, rename, switch between, and delete multiple independent Kanban projects. Each project retains its own isolated columns, cards, members, and activity log.

Drag & Drop Engine (Desktop + Mobile): Powered by @dnd-kit. Features PointerSensor, MouseSensor, and TouchSensor (with a 150ms delay and 5px tolerance). On mobile screens, columns stack vertically to eliminate horizontal scroll issues.

Security & RBAC Enforcement: Backend endpoints enforce permission checks (owner vs member vs viewer). Card mutations verify user membership before executing SQLite updates.

Real-Time WebSockets Sync: When multiple users collaborate on the same project, board updates (card moves, edits, deletions) are instantly broadcast via WebSockets (/ws/projects/{id}).

AI Kanban Assistant: Embedded chat drawer enables users to manage their board with natural language (e.g. 'Move high priority tasks to In Progress').

Activity Log & Notification System: Every project action (adding cards, editing titles, changing priority) is logged in SQLite and presented in interactive modal dialogs.

5. Complete Testing Methodology (80 Automated Tests)

The application is validated by an automated test suite comprising **80 total tests** across 3 distinct test runners:

- **Pytest (22 Tests):** Validates backend REST routes, SQLite database schemas, PBKDF2 password hashing, RBAC permission checks, and AI response structures.
- **Vitest (44 Tests):** Unit tests frontend helper utilities (filterAndSortBoard, moveCard, useUndoRedo) and React UI components (LoginForm, TaskFilterToolbar, ProjectSwitcher).
- **Playwright E2E (14 Tests):** Automated end-to-end browser tests in Headless Chromium. Verifies full user sign-in, task creation, dragging cards across columns, mobile viewport responsiveness, and multi-user login workflows.

6. DevOps Architecture: Docker, Render & Vercel

Why Docker?

Docker packages the Python runtime, FastAPI dependencies, Uvicorn server, and SQLite database into a standardized, lightweight Linux container. This eliminates 'it works on my machine' issues and ensures identical execution across local dev and cloud servers.

Render Cloud Backend Hosting:

Render reads the root render.yaml and builds the Docker container. It exposes port 8000 and serves CORS-enabled

API requests. Note: Render free instances spin down after 15 minutes of inactivity; the initial request takes ~30 seconds for cold-start initialization.

Vercel Frontend Hosting & Environment Variables:

Vercel hosts the Next.js static production bundle on an Edge CDN. The environment variable `NEXT_PUBLIC_API_URL` points to the Render backend URL (<https://drag-n-drop-28p3.onrender.com>). Because Next.js inlines `NEXT_PUBLIC_` variables at build time, updating this variable triggers a fresh Vercel rebuild to embed the live endpoint into the web app.